UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA
Proyecto Integral de Ingeniería de Software

Documentación del Sprint 3

Sistema Inteligente para la Gestión y Priorización de Incidencias de Soporte Técnico (SIGPI)

Casos de uso implementados · Fase RUP: Construcción I

Derivado del Tablero Kanban v3 y de las Historias de Usuario v3 · FastAPI en capas · React + TypeScript

EQUIPO
Haide Sánchez Gutiérrez · Laura Valeria Rodríguez Morales
Jesús Iniesta Valverde · Emmanuel Bernal Espejel

DOCENTE
Silvia Edith Albarrán Trujillo

METODOLOGÍA
Modelo de proceso: RUP (Proceso Unificado) | Seguimiento: Scrum

Toluca de Lerdo, Estado de México · 18 de octubre de 2026

Contenido

1. Objetivo y alcance del Sprint 3 3

2. Sprint Backlog y estado del tablero 4

3. Modelo de casos de uso del sprint 5

4. Arquitectura aplicada en los casos de uso 5

5. Casos de uso realizados 7

5.1 CU-S3-01 · Iniciar sesión (HU-01.1) 7

5.2 CU-S3-02 · Registrar una solicitud desde el portal (HU-01.1) 8

5.3 CU-S3-03 · Registro asistido y solicitante por verificar (HU-01.3) 9

5.4 CU-S3-04 · Calcular la prioridad (HU-02.3) 10

5.5 CU-S3-05 · Atender, resolver y confirmar (HU-04.1) 11

5.6 CU-S3-06 · Suspender y reanudar el reloj del SLA (HU-04.2) 12

5.7 Soporte transversal: errores y registro de eventos (S3-01) 14

6. Pruebas y calidad 15

7. Trazabilidad: historia → tarjeta → código → prueba 15

8. Avance y carga del sprint 16

9. Observaciones y ajustes para el Sprint 4 16

Referencias 17

1. Objetivo y alcance del Sprint 3

El Sprint 3 corresponde a la fase de Construcción I del Proceso Unificado. Su objetivo, fijado en el Sprint Backlog 3 (tarjeta S3-18), fue entregar el primer incremento funcional del sistema: el flujo de alta de solicitudes con inicio de sesión local (HU-01.1), el registro asistido con la contingencia de solicitante no localizado (HU-01.3) y el seguimiento con bitácora (HU-04.1), además del cálculo centralizado de la prioridad (HU-02.3) y la suspensión del reloj del SLA (HU-04.2).

Este documento describe los seis casos de uso trabajados durante el sprint. Para cada uno presenta su ficha (actor, precondiciones, flujos y postcondiciones), su relación con las historias de usuario y las reglas de negocio, la implementación por capas con fragmentos del código fuente y las pruebas automatizadas que verifican los criterios de aceptación. Al final se registran la trazabilidad, el avance del sprint y los ajustes para el Sprint 4. No incluye capturas de pantalla.

5 / 5	**14 / 17**	**236**	**31**
Historias del sprint con tarjetas entregadas	Tarjetas de desarrollo integradas (75 de 91 h)	Pruebas de backend aprobadas (pytest)	Pruebas de frontend aprobadas (Vitest)

Dato	Valor
Fase RUP	Construcción I (Sprint 3 de 5)
Capacidad planeada	97 h (91 h de desarrollo y 6 h de gestión), ≈ 12.1 h por integrante por semana
Periodo con actividad en el repositorio	Pull requests #10 a #17, fusionados entre el 15 y el 18 de octubre de 2026
Corte de este documento	18 de octubre de 2026, rama main en el commit b0847c0
Roles Scrum	Product Owner: Haide Sánchez · Scrum Master: Emmanuel Bernal · Developers: todo el equipo
Fuentes	Tablero Kanban SIGPI v3 · Historias de Usuario SIGPI v3 · Sprint Backlog 3 · Documentación del Sprint 2 · repositorio del proyecto

Tabla 1. Datos generales del Sprint 3.
Elaboración propia.

1.1 Historias de usuario comprometidas

Historia	Título	Actor	Prioridad · tamaño	Tarjetas S3	Caso de uso	Resultado
HU-01.1	Registrar una solicitud desde el portal	ACT-01	Debe · M	S3-02, S3-03, S3-04, S3-05	CU-S3-01 y CU-S3-02	Realizada
HU-01.3	Registrar una solicitud recibida por teléfono o en persona	ACT-02	Debe · S	S3-08, S3-09, S3-10, S3-11	CU-S3-03	Servidor realizado; interfaz y pruebas en S4
HU-02.3	Calcular la prioridad con la matriz urgencia-impacto	ACT-04	Debe · S	S3-06, S3-07	CU-S3-04	Realizada
HU-04.1	Registrar el avance en una bitácora única	ACT-02 · ACT-03	Debe · M	S3-12, S3-13, S3-14	CU-S3-05	Servidor realizado; detalle en interfaz en S4
HU-04.2	Suspender el reloj del SLA por causas ajenas	ACT-02 · ACT-03	Debe · S	S3-15, S3-16, S3-17	CU-S3-06	Realizada

Tabla 2. Historias del Sprint 3, caso de uso que las documenta y resultado. La tarjeta S3-01 (errores y registro de eventos) es transversal y no deriva de una historia.
Elaboración propia, con base en Equipo SIGPI (2026b, 2026c).

2. Sprint Backlog y estado del tablero

Una tarjeta se considera entregada cuando su producto está integrado en la rama main y, en las tarjetas de servidor e interfaz, cuenta con pruebas automatizadas (Definición de Terminado, Historias de Usuario v3). La tabla 3 registra el estado de las 22 tarjetas al corte del documento; las horas son las planeadas en el Sprint Backlog 3 (Equipo SIGPI, 2026a).

Tarjeta	Actividad	Responsable	Horas	Estado	Evidencia
S3-01	Manejo de errores y registro de eventos	Laura Rodríguez	6	Entregada	PR #17
S3-02	Formulario de registro de solicitud	Emmanuel Bernal	6	Entregada	PR #14
S3-03	Inicio de sesión con JWT y roles	Haide Sánchez	6	Entregada	PR #10
S3-04	Pantalla de inicio de sesión y rutas protegidas	Jesús Iniesta	6	Entregada	main
S3-05	Pruebas del alta de solicitud	Jesús Iniesta	4	Entregada	main
S3-06	Servicio de cálculo de prioridad	Emmanuel Bernal	6	Entregada	PR #12
S3-07	Pruebas de las nueve combinaciones de la matriz	Jesús Iniesta	4	Entregada	main
S3-08	Endpoint de registro asistido	Haide Sánchez	6	Entregada	PR #11
S3-09	Pantalla de registro asistido	Laura Rodríguez	6	Pendiente	—
S3-10	Solicitante no localizado: marca «por verificar»	Haide Sánchez	5	Entregada	PR #15
S3-11	Pruebas del registro asistido y la contingencia	Laura Rodríguez	4	Pendiente	Parcial: ver 5.3
S3-12	Servicio de seguimiento y resolución	Haide Sánchez	6	Entregada	PR #16
S3-13	Detalle de la solicitud con su bitácora	Laura Rodríguez	6	Pendiente	—
S3-14	Pruebas de resolución y confirmación	Jesús Iniesta	4	Entregada	main
S3-15	Estado «En espera» y reloj del SLA	Emmanuel Bernal	6	Entregada	PR #13
S3-16	Motivo de espera en la pantalla de atención	Jesús Iniesta	6	Entregada	main
S3-17	Pruebas de suspensión y reanudación del reloj	Emmanuel Bernal	4	Entregada	PR #13
S3-18	Planeación del Sprint 3	Emmanuel Bernal	1	Entregada	Sprint Backlog 3
S3-19	Estrategia de ramas y revisión de pull requests	Laura Rodríguez	2	Pendiente	—
S3-20	Revisión de riesgos del sprint 3	Haide Sánchez	1	Pendiente	—
S3-21	Demostración del incremento (Sprint 3)	Emmanuel Bernal	1	En elaboración	Notas de la demostración
S3-22	Retrospectiva 3 y velocidad	Jesús Iniesta	1	Pendiente	—

Tabla 3. Estado de las tarjetas del Sprint 3 al 18 de octubre de 2026. Evidencia: pull request fusionado o producto en la rama main. Elaboración propia, con base en Equipo SIGPI (2026a) y el historial del repositorio.

3. Modelo de casos de uso del sprint

Los casos de uso del sprint provienen de los casos de uso del sistema del Modelado de Negocios (SCU-01, SCU-02 y SCU-04). Cada historia de usuario concreta un escenario de uno de ellos.

Caso de uso	Nombre	Caso del sistema	Historia	Reglas	Estado
CU-S3-01	Iniciar sesión	SCU-01	HU-01.1	RN-01	Implementado
CU-S3-02	Registrar una solicitud desde el portal	SCU-01	HU-01.1	RN-01, RN-02, RN-03, RN-05	Implementado
CU-S3-03	Registro asistido y solicitante por verificar	SCU-01	HU-01.3	RN-01	Implementado (servidor)
CU-S3-04	Calcular la prioridad	SCU-02	HU-02.3	RN-05	Implementado
CU-S3-05	Atender, resolver y confirmar una solicitud	SCU-04	HU-04.1	RN-10	Implementado (servidor)
CU-S3-06	Suspender y reanudar el reloj del SLA	SCU-04	HU-04.2	RN-11	Implementado

Tabla 4. Casos de uso del Sprint 3 y su origen en el Modelado de Negocios.
Elaboración propia, con base en Equipo SIGPI (2026b).

Estados de la solicitud. Los casos CU-S3-03, CU-S3-05 y CU-S3-06 mueven la solicitud entre los estados que muestra la figura 1. Las reglas de transición viven en `domain/estados.py` y no consultan la base de datos.

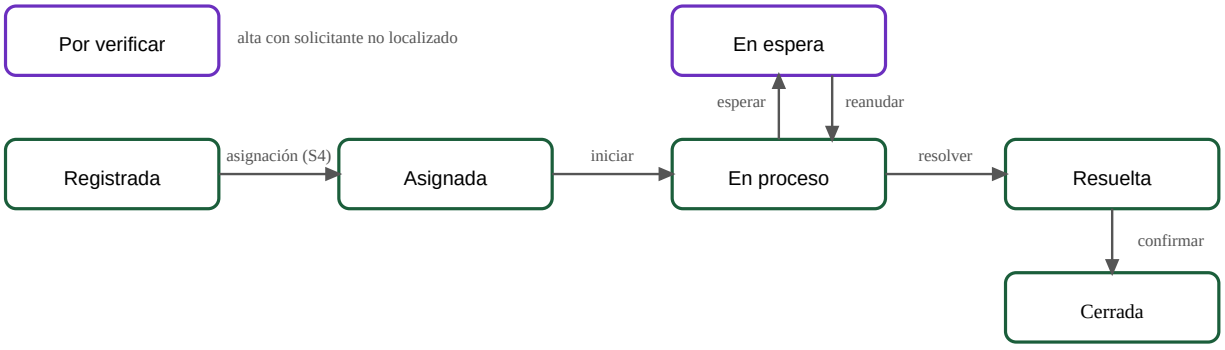

Figura 1. Estados de la solicitud y transiciones implementadas en el Sprint 3. En morado, los estados añadidos en el sprint («Por verificar» y «En espera»). La asignación se implementa en el Sprint 4.
Elaboración propia, con base en `domain/estados.py`.

4. Arquitectura aplicada en los casos de uso

Los casos de uso siguen la arquitectura en capas definida en el ADR del Sprint 1. La regla de dependencia es única: cada capa conoce solo a la inmediatamente inferior, y el dominio no depende de ninguna. Las reglas de negocio viven en los servicios y en el dominio, de modo que se prueban sin servidor web.

Capa	Paquete	Responsabilidad en el Sprint 3
Presentación (API)	api/routers/auth.py, solicitudes.py, seguimiento.py	Valida con esquemas Pydantic, exige el permiso del rol y traduce las excepciones del servicio a códigos HTTP (401, 403, 404, 409, 422).
Esquemas	schemas/auth.py, request.py	Contratos de entrada y salida; el alta rechaza indicar a la vez dos referencias al solicitante.

Capa	Paquete	Responsabilidad en el Sprint 3
Aplicación (servicios)	services/auth_service.py, solicitud_service.py, prioridad_service.py, seguimiento_service.py, sla_clock_service.py	Orquestan cada caso de uso y aplican las reglas RN-xx.
Dominio	domain/priority.py, domain/estados.py	Matriz urgencia-impacto y reglas puras de transición de estados.
Acceso a datos	repositories/auth_repository.py, models/request.py, alembic/	Consultas asíncronas; migraciones del diagnóstico y la solución, y de las pausas del SLA.
Integración	integrations/directorio_simulado.py	Directorio simulado (tabla local con datos de prueba) que sustituye al directorio institucional.
Transversal	core/security.py, exceptions.py, logging.py	JWT y roles, formato uniforme de error y registro estructurado de eventos.
Frontend	features/auth, features/requests	Páginas React + TypeScript con TanStack Query; contexto de sesión y rutas protegidas.

Tabla 5. Capas que intervienen en los casos de uso del Sprint 3. Elaboración propia.

5. Casos de uso realizados

5.1 CU-S3-01 · Iniciar sesión (HU-01.1)

Historia de usuario	HU-01.1 · Como usuario solicitante, quiero registrar una solicitud con mis credenciales institucionales. Este caso cubre la autenticación que la historia requiere (AC3).
Actor principal	Usuario solicitante (ACT-01); también lo usan los técnicos y administradores.
Tarjetas	S3-03 (servidor) y S3-04 (pantalla de inicio de sesión y rutas protegidas).
Precondiciones	El usuario tiene una cuenta con contraseña (se solicita al dar de alta al usuario, CU-S2-01).
Flujo principal	1. El usuario envía correo y contraseña. 2. El sistema busca la cuenta por correo y verifica la contraseña contra su hash. 3. Verifica que la cuenta esté activa. 4. Reinicia el contador de intentos fallidos. 5. Emite un token JWT con los roles del usuario y lo coloca en una cookie. 6. La interfaz guarda la sesión y habilita las rutas protegidas.
Flujos alternos	A1. Credenciales inválidas → HTTP 401 con el número de intentos fallidos. A2. Al llegar a tres intentos seguidos (`max_failed_login_attempts`) la respuesta indica que se ofrezca la recuperación de contraseña. A3. Cuenta desactivada → HTTP 403. A4. Correo inexistente → mismo mensaje y mismo costo de cómputo que una contraseña errónea, para no revelar qué cuentas existen.
Postcondiciones	Existe una sesión autenticada; el cierre de sesión (`POST /auth/logout`) elimina la cookie.
Endpoints	POST /auth/login · POST /auth/logout · GET /auth/me

Criterio	Enunciado	Estado y evidencia
AC3	Dado que mis credenciales son inválidas, cuando intento autenticarme tres veces seguidas, entonces el sistema me ofrece el procedimiento de recuperación de contraseña.	Cumplido en el servicio: la respuesta 401 incluye `offer_password_recovery`. Pendiente: pantalla de recuperación y prueba específica de los tres intentos (sección 9).

Tabla 6. Criterio de aceptación de HU-01.1 que corresponde a CU-S3-01.
Elaboración propia, con base en Equipo SIGPI (2026c).

`backend/app/services/auth_service.py` líneas 34–60

```
34      async def login(self, email: str, password: str) -> tuple[str, User]:
35          user = await self.repository.get_by_email(email)
36
37          if user is None or not user.password_hash:
38              # Mismo costo de cómputo que una cuenta real; el mensaje no distingue el caso.
39              verify_password(password, DUMMY_PASSWORD_HASH)
40              raise InvalidCredentialsError(failed_attempts=0, offer_password_recovery=False)
41
42          if not verify_password(password, user.password_hash):
43              attempts = user.failed_login_attempts + 1
44              await self.repository.set_failed_attempts(user, attempts)
45              raise InvalidCredentialsError(
46                  failed_attempts=attempts,
47                  offer_password_recovery=attempts >= settings.max_failed_login_attempts,
48              )
49
50          if not user.is_active:
51              raise InactiveAccountError("La cuenta está desactivada.")
52
53          user_id = user.id
54          if user.failed_login_attempts:
55              await self.repository.set_failed_attempts(user, 0)
56
57          loaded = await self.repository.get_by_id(user_id)
58          assert loaded is not None
59          roles = [ur.role.name for ur in loaded.roles if ur.role is not None]
60          return create_access_token(user_id, roles), loaded
```

Figura 2. Servicio de inicio de sesión (S3-03).

5.2 CU-S3-02 · Registrar una solicitud desde el portal (HU-01.1)

Historia de usuario	HU-01.1 · Registrar una solicitud desde el portal.
Actor principal	Usuario solicitante (ACT-01)
Origen	SCU-01 · RN-01, RN-02, RN-03 · prioridad Debe · tamaño M
Tarjetas	S3-02 (formulario) y S3-05 (pruebas del alta). El servicio de alta proviene de S2-16.
Precondiciones	El usuario inició sesión (CU-S3-01).
Flujo principal	1. El usuario abre «Nueva solicitud». 2. Captura título, descripción, ubicación y, opcionalmente, categoría, urgencia e impacto. 3. Cada campo obligatorio se valida al salir de él y el error desaparece al corregirlo. 4. Al enviar, el sistema genera el folio consecutivo, fija el estado «Registrada» y calcula la prioridad (CU-S3-04). 5. La interfaz muestra el acuse con el folio.
Flujos alternos	A1. Campos vacíos → se señalan los campos incompletos y no se envía. A2. Sesión vencida (HTTP 401) → mensaje de sesión expirada y el formulario permanece abierto. A3. Otro error del servidor → se muestra su mensaje.
Postcondiciones	La solicitud queda registrada con un folio único (SIG-0001, SIG-0002…) y su prioridad.
Endpoint	POST /solicitudes

Criterio	Enunciado	Estado y evidencia
AC1	Dado que soy un usuario activo y autenticado, cuando capturo los datos y envío la solicitud, entonces se genera un folio único y consecutivo, el estatus «Registrada» y se muestra el acuse.	Cumplido: `test_alta_asigna_folio_y_estado_inicial`, `test_folio_es_consecutivo_*` y, en interfaz, «AC1: alta válida … muestra el acuse con el folio». Pendiente: disparar la clasificación (HU-02.1, Sprint 4).
AC2	Dado que la descripción es corta o la ubicación está vacía, cuando intento enviar, entonces se rechaza el envío y se señala el campo.	Cumplido para campos vacíos (backend y frontend). Ajuste: el mínimo de 30 caracteres de la descripción no está aplicado (sección 9).

Tabla 7. Criterios de aceptación de HU-01.1 y su avance.
Elaboración propia, con base en Equipo SIGPI (2026c).

`frontend/src/features/requests/use-crear-solicitud.ts` líneas 1–40

```
 1  import { useMutation } from '@tanstack/react-query'
 2  import {
 3    createRequest,
 4    type CreateRequestInput,
 5    type Request,
 6  } from '@/api/solicitudes'
 7
 8  /**
 9   * Registra una solicitud de soporte (HU-01.1).
10   * Expone la solicitud creada para mostrar el acuse con su folio.
11   */
12  export function useCrearSolicitud() {
13    const mutation = useMutation<Request, Error, CreateRequestInput>({
14      mutationFn: createRequest,
15    })
16
17    return {
18      crear: mutation.mutate,
19      solicitud: mutation.data ?? null,
20      isPending: mutation.isPending,
21      error: mutation.error?.message ?? null,
22    }
23  }
24
```

Figura 3. Hook que encapsula el envío del formulario y expone la solicitud creada (S3-02).

5.3 CU-S3-03 · Registro asistido y solicitante por verificar (HU-01.3)

Historia de usuario	HU-01.3 · Como técnico de Nivel 1, quiero registrar en nombre del usuario una solicitud recibida por teléfono o en persona, para que ninguna quede fuera del registro único (RN-01).
Actor principal	Técnico de Nivel 1 (ACT-02)
Tarjetas	S3-08 y S3-10 (servidor). S3-09 (pantalla) y S3-11 (pruebas del registro asistido) pasan al Sprint 4.
Precondiciones	El técnico tiene el permiso de registro asistido.
Flujo principal	1. El técnico envía los datos de la solicitud, el canal de origen (teléfono o presencial) y la referencia del solicitante. 2. El sistema busca al solicitante; si lo localiza, la solicitud queda «Registrada». 3. Genera el folio y calcula la prioridad. 4. Guarda la solicitud y su primera entrada de bitácora, con el técnico como autor, en una sola transacción.
Flujos alternos	A1. Solicitante no localizado → la solicitud se registra con el estado «Por verificar» y la bitácora lo explica (no aparece en el directorio, o el directorio lo reconoce pero aún no tiene cuenta). A2. Solicitante dado de baja → HTTP 422. A3. Datos que indican a la vez o ninguna referencia al solicitante → el esquema los rechaza.
Postcondiciones	La solicitud tiene la misma trazabilidad que un registro directo y conserva el identificador institucional capturado.
Endpoint	POST /solicitudes/asistido

Criterio	Enunciado	Estado y evidencia
AC1	Dado que un usuario reporta por un canal no electrónico, cuando capturo la solicitud a su nombre indicando el canal, entonces se genera folio y se registra con la misma trazabilidad.	Cumplido en servidor: `test_asistido_localizado_deja_la_bitacora_como_registrada`. Pendiente: pantalla (S3-09).
AC2	Dado que el directorio no está disponible, cuando registro de forma asistida, entonces se guarda con la identidad declarada y la marca «por verificar».	Cumplido: `test_asistido_no_localizado_queda_por_verificar_y_lo_explica`.

Tabla 8. Criterios de aceptación de HU-01.3 y su avance.
Elaboración propia, con base en Equipo SIGPI (2026c).

`backend/app/services/solicitud_service.py` líneas 44–58

```
44      async def create_assisted_request(
45          self, payload: RequestAssistedCreate, registered_by_id: uuid.UUID
46      ) -> Request:
47          """Register a request received by phone or in person on behalf of a requester.
48
49          The technician who captures it is stored as the author of the first bitácora
50          entry, so the request and its origin are saved in a single transaction.
51          """
52          log = RequestLog(
53              author_id=registered_by_id,
54              previous_status=None,
55              action=f"Solicitud registrada {_CHANNEL_LABELS[payload.channel]}",
56              notes="Registro asistido: el técnico capturó la solicitud en nombre del solicitante.",
57          )
58          return await self._register(payload, log)
```

Figura 4. Registro asistido con su primera entrada de bitácora (S3-08).

`backend/app/domain/estados.py` líneas 28–36

```
28  def estado_inicial(solicitante_localizado: bool) -> RequestStatus:
29      """Estado con el que se registra una solicitud nueva.
30
31      Un solicitante no localizado no impide el registro: la solicitud queda
32      pendiente de verificación hasta que se confirme quién es.
33      """
34      if solicitante_localizado:
35          return RequestStatus.REGISTERED
36      return RequestStatus.PENDING_VERIFICATION
```

Figura 5. Regla del estado inicial según el solicitante sea localizado (S3-10).

5.4 CU-S3-04 · Calcular la prioridad (HU-02.3)

Historia de usuario	HU-02.3 · Como jefe del Departamento de TIC, quiero que la prioridad se calcule con la matriz urgencia-impacto y que nadie pueda fijarla manualmente, para que el orden de atención sea objetivo (RN-05).
Actor principal	Jefe del Departamento de TIC (ACT-04); el cálculo lo ejecuta el sistema.
Tarjetas	S3-06 (servicio) y S3-07 (pruebas de las nueve combinaciones).
Flujo principal	1. Al registrar una solicitud con urgencia e impacto, el servicio consulta la matriz del dominio. 2. Asigna la prioridad P1 a P4 que corresponde. 3. Toda solicitud pasa por este servicio; no existe otro punto donde se fije la prioridad.
Flujos alternos	A1. Falta la urgencia o el impacto → la prioridad queda vacía. A2. Valores fuera de 1 a 3 → el alta responde HTTP 422. A3. Intento de enviar la prioridad en el alta → el esquema la rechaza.
Postcondiciones	La prioridad es siempre función de la urgencia y el impacto.

Urgencia \ Impacto	Impacto 1	Impacto 2	Impacto 3
Urgencia 3	P2	P1	P1
Urgencia 2	P3	P2	P1
Urgencia 1	P4	P3	P2

Tabla 9. Matriz urgencia-impacto implementada en `domain/priority.py`*.*
Elaboración propia.

Criterio	Enunciado	Estado y evidencia
AC1	Dado que una solicitud tiene urgencia e impacto, cuando el sistema calcula la prioridad, entonces obtiene el valor P1 a P4 de la matriz.	Cumplido: `tests/domain/test_priority_matrix.py` (nueve combinaciones) y `test_alta_calcula_la_prioridad_con_la_matriz`.
AC2	Dado que soy usuario de cualquier rol, cuando intento editar directamente la prioridad, entonces el sistema no lo permite.	Cumplido: `test_alta_rechaza_la_edicion_directa_de_la_prioridad`. La modificación con justificación llegará con la reclasificación (Sprint 4).

Tabla 10. Criterios de aceptación de HU-02.3 y su avance.
Elaboración propia, con base en Equipo SIGPI (2026c).

`backend/app/services/prioridad_service.py` líneas 17–30

```
17  class PrioridadService:
18      def calcular(self, urgency: int | None, impact: int | None) -> Priority | None:
19          """Devuelve la prioridad, o `None` si aún falta la urgencia o el impacto."""
20          if urgency is None or impact is None:
21              return None
22          try:
23              return calculate_priority(urgency, impact)
24          except ValueError as exc:
25              raise PrioridadInvalidaError(str(exc)) from exc
26
27      def aplicar(self, request: Request) -> Priority | None:
28          """Calcula la prioridad con los valores de la solicitud y la asigna."""
29          request.priority = self.calcular(request.urgency, request.impact)
30          return request.priority
```

Figura 6. Servicio de cálculo de prioridad (S3-06).

5.5 CU-S3-05 · Atender, resolver y confirmar (HU-04.1)

Historia de usuario	HU-04.1 · Como técnico de soporte, quiero registrar el diagnóstico, la solución aplicada y los cambios de estatus en una bitácora única, para dejar constancia y dar visibilidad del avance.
Actor principal	Técnico de soporte (ACT-02 · ACT-03); confirma el solicitante o un rol autorizado.
Tarjetas	S3-12 (servicio) y S3-14 (pruebas de resolución y confirmación). S3-13 (detalle con bitácora en la interfaz) pasa al Sprint 4.
Precondiciones	La solicitud está asignada al técnico.
Flujo principal	1. El técnico inicia la atención: la solicitud pasa a «En proceso» y se marca la primera respuesta. 2. Registra el diagnóstico; cada versión queda en la bitácora. 3. Resuelve la solicitud con la solución aplicada: el sistema calcula el tiempo efectivo de resolución sin contar las esperas y anota que se solicita confirmación. 4. El solicitante confirma y la solicitud pasa a «Cerrada». 5. Cada cambio de estado agrega a la bitácora marca de tiempo, autor, estado anterior y estado nuevo.
Flujos alternos	A1. Operación no permitida desde el estado actual → HTTP 409. A2. Solicitud inexistente → HTTP 404. A3. Falta el permiso del rol → HTTP 403.
Postcondiciones	La bitácora conserva todo el historial; solo admite altas.
Endpoints	POST /{id}/iniciar · /diagnostico · /resolver · /confirmar · GET /{id}/bitacora

Criterio	Enunciado	Estado y evidencia
AC1	Dado que la solicitud está asignada a mí, cuando registro el diagnóstico, aplico la solución y la marco «Resuelta», entonces el sistema calcula el tiempo de resolución, evalúa el cumplimiento, notifica al usuario y solicita su confirmación.	Cumplido: tiempo de resolución y solicitud de confirmación (`test_resolver_calcula_sla_al_resolver`). Pendiente: evaluar el cumplimiento contra el acuerdo vigente y notificar (HU-07.1, Sprint 4).
AC2	Dado que cambio el estatus de una solicitud, cuando guardo el cambio, entonces la bitácora registra marca de tiempo, autor y estatus anterior y nuevo.	Cumplido: `test_resolver_registra_bitacora` y `test_confirmar_registra_bitacora`.

Tabla 11. Criterios de aceptación de HU-04.1 y su avance.
Elaboración propia, con base en Equipo SIGPI (2026c).

backend/app/services/seguimiento_service.py líneas 106–135

```
106          request = await self._load(request_id)
107          if not puede_resolver(request.status):
108              raise EstadoInvalidoError(
109                  f"una solicitud en estado {request.status} no puede resolverse"
110              )
111          corte = ahora or _ahora()
112          previous = request.status
113          if diagnosis is not None:
114              request.diagnosis = diagnosis
115          request.solution = solution
116          request.status = RequestStatus.RESOLVED
117          request.resolved_at = corte
118          if request.first_response_at is None:
119              request.first_response_at = corte
120          request.resolution_minutes = round(minutos_efectivos(request, corte))
121          request.logs.append(
122              RequestLog(
123                  author_id=author_id,
124                  previous_status=previous,
125                  new_status=RequestStatus.RESOLVED,
126                  action="Solicitud resuelta",
127                  notes=(
128                      f"Solución aplicada: {solution}\n"
129                      f"Tiempo efectivo de resolución: {request.resolution_minutes} min.\n"
130                      "Se solicita al solicitante confirmar la solución."
131                  ),
132              )
133          )
134          await self._session.commit()
135          return await self._load(request_id)
```

Figura 7. Resolución de una solicitud con cálculo del tiempo efectivo (S3-12).

5.6 CU-S3-06 · Suspender y reanudar el reloj del SLA (HU-04.2)

Historia de usuario	HU-04.2 · Como técnico de soporte, quiero poner la solicitud «En espera» cuando depende de una refacción o de una acción del usuario y que el reloj se suspenda, para no incumplir el compromiso por causas ajenas (RN-11).
Actor principal	Técnico de soporte (ACT-02 · ACT-03)
Tarjetas	S3-15 (estado y reloj), S3-16 (motivo de espera en la interfaz) y S3-17 (pruebas).
Precondiciones	La solicitud está «Asignada» o «En proceso».
Flujo principal	1. El técnico elige «Marcar en espera» y el motivo (insumo externo, información del solicitante u otro con detalle). 2. El sistema pasa la solicitud a «En espera», abre un periodo de pausa con inicio, autor y motivo y registra la bitácora. 3. Al recibir el insumo, el técnico reanuda la atención. 4. El sistema cierra el periodo, regresa la solicitud a «En proceso» y anota el tiempo en espera. 5. El tiempo efectivo es el transcurrido menos la suma de los periodos en espera.
Flujos alternos	A1. Estado que no admite la operación → HTTP 409. A2. Solicitud inexistente → HTTP 404. A3. Motivo desconocido → HTTP 422. A4. Sin sesión → HTTP 401. A5. En la interfaz, el motivo «Otro» exige describirlo.
Postcondiciones	Cada espera queda como un periodo con inicio, fin y motivo.
Endpoints	POST /solicitudes/{id}/espera · POST /solicitudes/{id}/reanudar

Criterio	Enunciado	Estado y evidencia
AC1	Dado que la solución depende de una refacción o de un tercero, cuando cambio la solicitud a «En espera», entonces el cómputo del acuerdo de nivel de servicio se suspende.	Cumplido: `test_suspender_cambia_el_estado_y_registra_motivo_y_bitacora` y `test_una_espera_abierta_cuenta_hasta_el_momento_de_consulta`.

Criterio	Enunciado	Estado y evidencia
AC2	Dado que se recibe el insumo faltante, cuando reanudo la atención, entonces el cómputo se reanuda descontando el periodo suspendido.	Cumplido: `test_el_periodo_suspendido_se_descuenta_del_tiempo_efectivo` y `test_varias_esperas_se_acumulan`.

Tabla 12. Criterios de aceptación de HU-04.2 y su avance.
Elaboración propia, con base en Equipo SIGPI (2026c).

`backend/app/services/sla_clock_service.py` líneas 39–56

```
39  def minutos_suspendidos(request: Request, ahora: datetime | None = None) -> float:
40      """Suma los periodos de espera; el periodo abierto cuenta hasta `ahora`."""
41      corte = ahora or _ahora()
42      return sum(_minutos(p.started_at, p.ended_at or corte) for p in request.sla_pauses)
43
44
45  def minutos_efectivos(request: Request, ahora: datetime | None = None) -> float:
46      """Minutos que cuentan para el SLA: transcurridos desde el alta, sin las esperas."""
47      corte = ahora or _ahora()
48      transcurridos = _minutos(request.created_at, corte)
49      return max(transcurridos - minutos_suspendidos(request, corte), 0.0)
50
51
52  def minutos_restantes(
53      request: Request, resolution_minutes: int, ahora: datetime | None = None
54  ) -> float:
55      """Minutos que faltan para vencer la resolución; negativo si ya venció."""
56      return resolution_minutes - minutos_efectivos(request, ahora)
```

Figura 8. Cálculo del tiempo efectivo descontando los periodos en espera (S3-15).

`backend/app/services/sla_clock_service.py` líneas 84–110

```
84          request = await self._load(request_id)
85          if not puede_suspender(request.status):
86              raise EstadoInvalidoError(
87                  f"una solicitud en estado {request.status} no puede pasar a «En espera»"
88              )
89          corte = ahora or _ahora()
90          previous = request.status
91          request.status = RequestStatus.WAITING
92          request.sla_pauses.append(
93              RequestSlaPause(
94                  reason=reason,
95                  detail=detail,
96                  started_by_id=author_id,
97                  started_at=corte,
98              )
99          )
100         request.logs.append(
101             RequestLog(
102                 author_id=author_id,
103                 previous_status=previous,
104                 new_status=RequestStatus.WAITING,
105                 action="Reloj del SLA suspendido",
106                 notes=_nota(reason, detail),
107             )
108         )
109         await self._session.commit()
110         return await self._load(request_id)
```

Figura 9. Suspensión del reloj: estado, periodo de pausa y bitácora en una sola transacción (S3-15).

5.7 Soporte transversal: errores y registro de eventos (S3-01)

La tarjeta S3-01 no deriva de una historia, pero sostiene a todos los casos anteriores. Define una jerarquía de errores esperados del sistema (no encontrado, conflicto, regla de negocio, autenticación) y un manejador global que los convierte en respuestas con una forma uniforme: `detail`, `code` (identificador estable) y `request_id`, que permite ubicar el error en el registro estructurado de eventos. La prueba `tests/test_errores_y_logging.py` reúne 15 casos.

backend/app/core/exceptions.py — líneas 21–34

```
21  class SigpiError(Exception):
22      """Base de los errores esperados del sistema.
23
24      Los servicios pueden lanzar una subclase y el manejador global la convierte en una
25      respuesta HTTP, sin que cada router tenga que repetir `try/except` + `HTTPException`.
26      """
27
28      status_code: int = status.HTTP_400_BAD_REQUEST
29      code: str = "sigpi_error"
30      default_message: str = "No se pudo completar la operación."
31
32      def __init__(self, message: str | None = None) -> None:
33          self.message = message or self.default_message
34          super().__init__(self.message)
```

Figura 10. Clase base de los errores esperados del sistema (S3-01).

6. Pruebas y calidad

Sobre la rama principal (commit b0847c0) se ejecutaron las pruebas del proyecto. El pipeline de integración continua ejecuta ruff, pytest, el compilador de TypeScript, ESLint, Prettier y Vitest en cada pull request.

Conjunto	Herramienta	Resultado
Backend	pytest	236 pruebas aprobadas y 2 omitidas (requieren PostgreSQL).
Frontend	Vitest + Testing Library	31 pruebas aprobadas en 4 archivos.

Tabla 13. Resultado de las pruebas sobre la rama principal.
Elaboración propia, a partir de la ejecución del 18 de octubre de 2026.

Caso de uso	Archivo de pruebas	Pruebas
CU-S3-01	backend/tests/api/test_usuarios.py (inicio de sesión tras el alta)	Parcial
CU-S3-02	backend/tests/api/test_solicitudes.py	13
CU-S3-02	frontend/src/features/requests/solicitud-form.test.tsx	8
CU-S3-03	backend/tests/api/test_solicitante_por_verificar.py	8
CU-S3-04	backend/tests/domain/test_priority_matrix.py · tests/services/test_prioridad_service.py	12 · 6
CU-S3-05	backend/tests/services/test_seguimiento.py · test_seguimiento_service.py · tests/domain/test_estados_seguimiento.py	13 · 8 · 4
CU-S3-06	backend/tests/services/test_sla_clock.py	13
CU-S3-06	frontend/src/features/requests/components/EsperaDialog.test.tsx	7
S3-01	backend/tests/test_errores_y_logging.py	15

Tabla 14. Pruebas por caso de uso. Se cuentan las funciones de prueba de cada archivo.
Elaboración propia.

7. Trazabilidad: historia → tarjeta → código → prueba

Historia	Tarjetas	Código principal	Prueba
HU-01.1	S3-02, S3-03, S3-04, S3-05	routers/auth.py · services/auth_service.py · features/auth · features/requests/solicitud-form.tsx	solicitud-form.test.tsx · test_solicitudes.py
HU-01.3	S3-08, S3-10	routers/solicitudes.py (POST /asistido) · services/solicitud_service.py · integrations/directorio_simulado.py	test_solicitante_por_verificar.py
HU-02.3	S3-06, S3-07	services/prioridad_service.py · domain/priority.py	test_priority_matrix.py · test_prioridad_service.py
HU-04.1	S3-12, S3-14	routers/seguimiento.py · services/seguimiento_service.py · domain/estados.py	test_seguimiento.py · test_seguimiento_service.py
HU-04.2	S3-15, S3-16, S3-17	services/sla_clock_service.py · features/requests/components/EsperaDialog.tsx	test_sla_clock.py · EsperaDialog.test.tsx

Tabla 15. Trazabilidad de las historias del Sprint 3.
Elaboración propia.

8. Avance y carga del sprint

La tabla 16 compara las horas planeadas con las de las tarjetas integradas al corte. Las horas reales no se midieron; la comparación se hace con las horas planeadas y la retrospectiva (S3-22) registrará la velocidad.

Integrante	Tarjetas asignadas	Horas planeadas	Horas de tarjetas integradas	Tarjetas pendientes
Haide Sánchez	5	24	23	S3-20
Laura Rodríguez	5	24	6	S3-09, S3-11, S3-13, S3-19
Jesús Iniesta	6	25	24	S3-22
Emmanuel Bernal	6	24	23	S3-21 (en elaboración)
Total	22	97	76	—

Tabla 16. Horas planeadas y de tarjetas integradas por integrante, según el responsable del tablero. Elaboración propia, con base en Equipo SIGPI (2026a) y el historial del repositorio.

Al corte se integraron 14 de las 17 tarjetas de desarrollo (75 de 91 h) y la tarjeta de planeación (1 h). Las 3 tarjetas de desarrollo pendientes suman 16 h.

9. Observaciones y ajustes para el Sprint 4

Con base en el estado de la rama principal, el equipo propone los ajustes siguientes.

N.º	Observación	Ajuste propuesto
1	Tres tarjetas de desarrollo no se integraron: S3-09, S3-11 y S3-13 (16 h).	Retomarlas al inicio del Sprint 4 para completar las historias HU-01.3 y HU-04.1 en todas sus capas.
2	El criterio AC3 de HU-01.1 (recuperación tras tres intentos fallidos) está en el servicio, pero sin prueba específica ni pantalla de recuperación.	Agregar pruebas del contador de intentos y de la respuesta con `offer_password_recovery`.
3	La descripción de la solicitud exige hoy un mínimo de 1 carácter, no los 30 de AC2 de HU-01.1.	Alinear el esquema y el formulario con el criterio de aceptación.
4	La evaluación del cumplimiento del compromiso y la notificación al solicitante (AC1 de HU-04.1) dependen de historias del Sprint 4.	Enlazarlas con HU-07.1 y el acuerdo de nivel de servicio vigente.
5	Tres tarjetas de gestión siguen abiertas: S3-19, S3-20 y S3-22.	Cerrarlas con sus entregables (guía de contribución, RiskLog v3 y registro de velocidad).

Tabla 17. Observaciones del Sprint 3 y ajustes propuestos. Elaboración propia.

Referencias

Equipo SIGPI. (2026a). *Sprint Backlog 3* [Documento del proyecto]. Facultad de Ingeniería, Universidad Autónoma del Estado de México.

Equipo SIGPI. (2026b). *Modelado de negocios: Sistema Inteligente para la Gestión y Priorización de Incidencias de Soporte Técnico* [Documento del proyecto]. Departamento de TIC, Facultad de Ingeniería, Universidad Autónoma del Estado de México.

Equipo SIGPI. (2026c). *Historias de usuario SIGPI: Versión 3* [Documento del proyecto]. Facultad de Ingeniería, Universidad Autónoma del Estado de México.

Equipo SIGPI. (2026d). *Tablero Kanban del Proyecto: Versión 3* [Documento del proyecto]. Facultad de Ingeniería, Universidad Autónoma del Estado de México.

Equipo SIGPI. (2026e). *Documentación del Sprint 2* [Documento del proyecto]. Facultad de Ingeniería, Universidad Autónoma del Estado de México.